

HDOC DAY 39

PROBLEM 1

Write a C program containing a variadic function: `double weightedAverage(int count, ...)`. The function should accept "count" and pairs of values (each pair consists of a value and a weight). It should calculate and return the weighted average.

SOLUTION

```
#include <stdio.h>
```

```
#include <stdarg.h>
```

```
double weightedAverage(int count, ...)
```

```
{
```

```
    va_list args;
```

```
    double weightedSum = 0.0;
```

```
    double totalWeight = 0.0;
```

```
    va_start(args, count);
```

```
    for (int i = 0; i < count; i++) {
```

```
        double value = va_arg(args, double);
```

```
        double weight = va_arg(args, double);
```

```
        weightedSum += value * weight;
```

```
        totalWeight += weight;
```

```
    }
```

```
    va_end(args);
```

```
    if (totalWeight == 0.0)
        return 0.0;

    return weightedSum / totalWeight;
}

int main()
{
    double result;

    result = weightedAverage(
        3,
        10.0, 2.0,
        20.0, 3.0,
        30.0, 5.0
    );

    printf("Weighted Average = %.2f\n", result);

    return 0;
}
```

PROBLEM 2

1. **Write a C program that accepts any number of command-line arguments representing integers. The program should:**
 - a. Validate each argument and ignore non-integer values.
 - b. Process the valid integers and display the valid integers in ascending order.

- c. Display the minimum, maximum, average, median, and second-largest distinct value.
- d. Use appropriate user-defined functions for validation, sorting, and calculation.

SOLUTION

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <errno.h>

#include <limits.h>


/* Function declarations */

int isInteger(const char *str, int *value);

void sortArray(int arr[], int n);

double calculateAverage(int arr[], int n);

double calculateMedian(int arr[], int n);

int findSecondLargestDistinct(int arr[], int n, int *result);


int main(int argc, char *argv[])
{
    int *numbers;

    int count = 0;

    int value;


    if (argc < 2) {
        printf("Usage: %s <integer1> <integer2> ...\\n", argv[0]);
        return 1;
    }
```

```
/* Allocate memory for all possible arguments */
```

```
numbers = malloc((argc - 1) * sizeof(int));
```

```
if (numbers == NULL) {
```

```
    printf("Memory allocation failed.\n");
```

```
    return 1;
```

```
}
```

```
/* Validate and store only integer arguments */
```

```
for (int i = 1; i < argc; i++) {
```

```
    if (isInteger(argv[i], &value)) {
```

```
        numbers[count++] = value;
```

```
    }
```

```
}
```

```
if (count == 0) {
```

```
    printf("No valid integers were provided.\n");
```

```
    free(numbers);
```

```
    return 0;
```

```
}
```

```
/* Sort the valid integers */
```

```
sortArray(numbers, count);
```

```
/* Display valid integers */
```

```
printf("Valid integers in ascending order:\n");
```

```
for (int i = 0; i < count; i++) {
```

```

        printf("%d ", numbers[i]);
    }
    printf("\n");

    /* Display statistics */
    printf("Minimum: %d\n", numbers[0]);
    printf("Maximum: %d\n", numbers[count - 1]);
    printf("Average: %.2f\n", calculateAverage(numbers, count));
    printf("Median: %.2f\n", calculateMedian(numbers, count));

    /* Find second-largest distinct value */
    if (findSecondLargestDistinct(numbers, count, &value)) {
        printf("Second-largest distinct value: %d\n", value);
    } else {
        printf("Second-largest distinct value: Not available\n");
    }

    free(numbers);

    return 0;
}

/*
 * Validates whether a command-line argument is an integer.
 * If valid, the integer is stored in *value.
 */
int isInteger(const char *str, int *value)
{

```

```

char *end;

long number;

if (str == NULL || *str == '\0')

    return 0;

errno = 0;

number = strtol(str, &end, 10);

/* Check for conversion errors or integer overflow */
if (errno == ERANGE || number < INT_MIN || number > INT_MAX)

    return 0;

/* Make sure the entire string represents an integer */
if (*end != '\0')

    return 0;

*value = (int)number;

return 1;
}

```

```

/* Sorts the array using bubble sort */
void sortArray(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];

```

```
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
    }
}
}
```

/* Calculates the average */

```
double calculateAverage(int arr[], int n)
{
    long long sum = 0;

    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }

    return (double)sum / n;
}
```

/* Calculates the median */

```
double calculateMedian(int arr[], int n)
{
    if (n % 2 == 1) {
        return arr[n / 2];
    }

    return (arr[n / 2 - 1] + (double)arr[n / 2]) / 2.0;
}
```

```
/*  
 * Finds the second-largest distinct value.  
 * Returns 1 if found, 0 otherwise.  
 */  
int findSecondLargestDistinct(int arr[], int n, int *result)  
{  
    int largest = arr[n - 1];  
  
    for (int i = n - 2; i >= 0; i--) {  
        if (arr[i] != largest) {  
            *result = arr[i];  
            return 1;  
        }  
    }  
  
    return 0;  
}
```